

Programação Funcional em Haskell

Composição e tipos

REALIZAR APÓS posição 27: 8.2 - Programação Funcional em Haskell: Tipo de Dado Fold

Posições na playlist 17 a 27 | 120 minutos | 100 pontos

Curso principal: Programação Funcional em Haskell - Curso Completo - UFABC |
<https://www.youtube.com/watch?v=eTisiy5FB7k&list=PLYItvall0TqJ25sVTLcMhxsE0Hci58mpQ&index=1>

Conteúdo verificável: Alta ordem; folds; composição; type; ADTs; tipos recursivos; álgebra dos tipos; zipper; typeclasses; monoid; tipo Fold

Instruções

Resolva sem consultar o gabarito. Apresente desenvolvimento, hipóteses, unidades e verificações. Código pode ser escrito em papel ou editor sem sugestões automáticas; use somente recursos compatíveis com o bloco. Ferramentas de simulação são permitidas apenas onde solicitadas. Use folhas adicionais para diagramas e cálculos.

Pontuação: Q1 = 15; Q2 = 25; Q3 = 25; Q4 = 35. Crédito parcial depende de raciocínio demonstrado. Meta: ≥ 80 prosseguir; 70–79 revisar erros; < 70 revisar o bloco.

Aluno(a): _____ Data: _____

Alinhamento: títulos/ordem e informações públicas do curso. As aulas não foram integralmente assistidas; detalhes não expostos não estão certificados.

Alta ordem e folds

(a) Reescreva soma dos quadrados de lista usando map e foldr. (b) Compare foldr e foldl em subtração sobre [1, 2, 3], base0. (c) Explique foldl estrito e possível acúmulo de expressões pendentes.

Desenvolvimento / justificativa / esquema:

ADTs, aliases e typeclasses

Considere type Nome=String e data Resultado a=Falha String|Sucesso a. (a) Explique diferença entre alias e novo tipo algébrico. (b) Implemente transformaResultado f preservando falha. (c) Explique como Eq/Show dependem dos tipos armazenados numa derivação usual.

Desenvolvimento / justificativa / esquema:

Zipper de lista

Represente foco de lista por(esquerda invertida, direita iniciando no foco). (a) Dê representação de foco no3 em[1, 2, 3, 4]. (b) Defina mover à esquerda/direita. (c) Explique recuperação da lista e limites de movimento.

Desenvolvimento / justificativa / esquema:

Monoid e agregação

(a) Mostre que listas com concatenação e[] formam monoid. (b) Dê aplicação de agregação a soma e contagem em uma travessia, descrevendo estado, passo e finalização de um Fold. (c) Explique as leis necessárias para combinar resultados parciais corretamente.

Desenvolvimento / justificativa / esquema:

Gabarito e Resoluções

Abra esta seção somente depois de concluir a tentativa. Há mais de um método correto possível; compare também o processo e os casos extremos.

Critérios de correção

Q1: 4 pontos pela modelagem/conceito, 8 pela solução, 3 pela verificação. Q2 e Q3: 6 pela modelagem, 14 pela solução, 5 pela justificativa/verificação. Q4: 8 pela modelagem, 20 pela solução, 7 pela análise crítica e casos extremos. Nos itens dissertativos, a solução inclui os conceitos e relações obrigatórios; crédito integral exige conectar esses conceitos ao cenário. Em código, valorize correção do contrato, casos base e explicação; uma chamada a biblioteca que evita a implementação pedida não substitui o algoritmo.

Questão 1 - Alta ordem e folds (15 pontos)

Soma: $\text{foldr } (+) \ 0 \ (\text{map } (\backslash x \rightarrow x * x) \ xs)$. $\text{foldr } (-) \ 0 \ [1, 2, 3] = 1 - (2 - (3 - 0)) = 2$; $\text{foldl } (-) \ 0 \ [1, 2, 3] = ((0 - 1) - 2) - 3 = -6$. foldl pode acumular thunks no acumulador; versão estrita força o acumulador segundo sua definição. Associatividade e avaliação importam; não afirmar que folds são intercambiáveis para qualquer operador.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Questão 2 - ADTs, aliases e typeclasses (25 pontos)

Nome é sinônimo, não cria isolamento nominal. Resultado tem construtores e alternativas; $\text{transformaResultado } f(\text{Falha } e) = \text{Falha } e$; $\text{transformaResultado } f(\text{Sucesso } x) = \text{Sucesso } (f\ x)$. Derivar Eq/Show exige instâncias adequadas para a , pois comparação/exibição de Sucesso usa o valor armazenado. Pattern matching deve cobrir ambos os casos; não tentar extrair valor de uma falha sem política.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Questão 3 - Zipper de lista (25 pontos)

Representação $([2, 1], [3, 4])$. Direita: $(ls, x:rs) \rightarrow (x:ls, rs)$; esquerda: $(x:ls, rs) \rightarrow (ls, x:rs)$. Lista original = reverse $ls ++ rs$. Movimento é $O(1)$; no extremo correspondente não há elemento para mover, deve sinalizar ausência ou manter conforme contrato. Guardar esquerda invertida evita percorrê-la a cada deslocamento.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Questão 4 - Monoid e agregação (35 pontos)

Concatenação associativa; $[]$ é identidade à esquerda/direita. Estado(soma, contagem), inicial(0, 0), passo(s, n), $x \rightarrow (s+x, n+1)$, finalização devolve par ou média com caso vazio. Combinar dois estados por soma componente a componente é associativo e tem identidade(0, 0), permitindo fusão de partes. Não usar média simples das médias sem ponderar contagens. O exercício descreve o Fold abstratamente, sem exigir uma representação interna não verificada do curso.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Diagnóstico do checkpoint

Registre sua nota e os conceitos a revisar em progresso.csv. Antes de avançar, refaça os itens errados sem olhar a resolução e crie um pequeno exemplo novo que use a mesma ideia. Nota alta sem justificar respostas não demonstra domínio.